

Nome do Integrante 1, Nome do Integrante 2, Nome do Integrante 3

**Projeto Computacional #3:
Resolução Numérica da Dinâmica Não-Linear
de um Pêndulo Simples**

São Carlos

2026

Nome do Integrante 1, Nome do Integrante 2, Nome do Integrante 3

Projeto Computacional #3:
Resolução Numérica da Dinâmica Não-Linear
de um Pêndulo Simples

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação
Disciplina SME0104 — Cálculo Numérico
Prof. Geovane Augusto Haveroth

São Carlos
2026

SUMÁRIO

1 INTRODUÇÃO

O pêndulo simples é um dos sistemas físicos mais estudados na mecânica clássica, servindo como modelo fundamental para fenômenos oscilatórios. Embora sua equação diferencial não-linear seja bem conhecida, soluções analíticas fechadas em termos de amplitude arbitrária não existem, tornando a abordagem numérica indispensável.

Neste trabalho, o problema é formulado como um **Problema de Valor de Contorno (PVC)**: dado o intervalo de tempo $t \in (0, T)$ com $T = 2\pi$, deseja-se determinar a trajetória angular $\theta(t)$ que satisfaça simultaneamente a equação diferencial governante e as condições de contorno $\theta(0) = \alpha$ e $\theta(T) = \beta$.

A estratégia de resolução combina dois métodos clássicos de cálculo numérico:

1. **Método de Diferenças Finitas** para discretizar a equação diferencial e converter o PVC em um sistema algébrico não-linear;
2. **Método de Newton-Raphson** para resolver iterativamente esse sistema.

São investigados, ainda, a influência do chute inicial na convergência do método iterativo, a comparação entre o modelo não-linear e sua aproximação linearizada para pequenos ângulos, e um estudo paramétrico contrastando o comportamento do pêndulo na superfície terrestre e na superfície de Mercúrio.

2 DESENVOLVIMENTO

2.1 MODELAGEM MATEMÁTICA

2.1.1 Equação Diferencial Governante

A equação de movimento de um pêndulo simples de comprimento L , sob aceleração gravitacional g , sem amortecimento e sem forçamento externo, é dada por:

$$\theta''(t) + \frac{g}{L} \sin(\theta(t)) = 0, \quad t \in (0, T), \quad (2.1)$$

com condições de contorno:

$$\theta(0) = \alpha, \quad \theta(T) = \beta. \quad (2.2)$$

Os parâmetros físicos e computacionais adotados são listados na Tabela ??.

Tabela 1 – Parâmetros físicos e computacionais do problema.

Parâmetro	Símbolo	Valor	Unidade
Intervalo de tempo	T	2π	—
Comprimento da haste	L	1,0	m
Gravidade (Terra)	g	9,8	m/s ²
Gravidade (Mercúrio)	g_{mer}	3,7	m/s ²
Ângulo inicial	α	0,7	rad
Ângulo final	β	0,7	rad
Pontos internos da malha	m	100 ou 1000	—
Tolerância de convergência	ε_{tol}	10^{-10}	—

Fonte: Elaborado pelos autores.

2.1.2 Método de Diferenças Finitas

O intervalo $[0, T]$ é discretizado em $m + 2$ pontos equiespaçados:

$$t_i = i h, \quad i = 0, 1, \dots, m + 1, \quad h = \frac{T}{m + 1}, \quad (2.3)$$

onde $t_0 = 0$ e $t_{m+1} = T$ correspondem aos nós de contorno, e $t_1, \dots, t_m$ são os m nós internos onde θ é incógnita.

Aproximando a derivada de segunda ordem por diferenças finitas centradas de segunda ordem:

$$\theta''(t_i) \approx \frac{\theta_{i-1} - 2\theta_i + \theta_{i+1}}{h^2}, \quad (2.4)$$

a equação (??) discretizada em cada nó interno $i = 1, \dots, m$ resulta em:

$$G_i(\boldsymbol{\theta}) = \theta_{i-1} - 2\theta_i + \theta_{i+1} + h^2 \frac{g}{L} \sin(\theta_i) = 0, \quad (2.5)$$

incorporando $\theta_0 = \alpha$ em G_1 e $\theta_{m+1} = \beta$ em G_m . O conjunto das m equações (??) forma o sistema não-linear $\mathbf{G}(\boldsymbol{\theta}) = \mathbf{0}$.

2.1.3 Jacobiana Analítica

A matriz Jacobiana $J(\boldsymbol{\theta}) = \partial \mathbf{G} / \partial \boldsymbol{\theta} \in \mathbb{R}^{m \times m}$ é obtida diferenciando (??):

$$J_{i,j} = \begin{cases} -2 + h^2 \frac{g}{L} \cos(\theta_i), & j = i \\ 1, & j = i \pm 1 \\ 0, & \text{caso contrário.} \end{cases} \quad (2.6)$$

A estrutura resultante é **tridiagonal**, o que é computacionalmente vantajoso pois permite resolver cada sistema linear $J \Delta \boldsymbol{\theta} = -\mathbf{G}$ em $\mathcal{O}(m)$ operações.

2.1.4 Método de Newton-Raphson

Dado um vetor inicial $\boldsymbol{\theta}^{(0)}$, as iterações de Newton-Raphson seguem o esquema:

$$J(\boldsymbol{\theta}^{(k)}) \Delta \boldsymbol{\theta}^{(k)} = -\mathbf{G}(\boldsymbol{\theta}^{(k)}), \quad \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \Delta \boldsymbol{\theta}^{(k)}. \quad (2.7)$$

O critério de parada adotado é o **erro relativo**:

$$\varepsilon_r^{(k)} = \frac{\|\Delta \boldsymbol{\theta}^{(k)}\|}{\|\boldsymbol{\theta}^{(k+1)}\|} < \varepsilon_{\text{tol}} = 10^{-10}. \quad (2.8)$$

2.2 IMPLEMENTAÇÃO COMPUTACIONAL

O código foi escrito em Python 3, utilizando NumPy para álgebra linear e Matplotlib para visualização. A arquitetura segue a modularização recomendada no roteiro:

```

1 def montar_sistema(theta, h, L, g, alpha, beta):
2     """Computa G(theta) via diferenças finitas centradas."""
3     m = len(theta); h2 = h**2; G = np.zeros(m)
4     for i in range(m):
5         tp = alpha if i == 0 else theta[i-1]
6         tn = beta if i == m-1 else theta[i+1]
7         G[i] = tp - 2*theta[i] + tn + h2*(g/L)*np.sin(theta[i])
8     return G
9
10 def calcular_jacobiana(theta, h, L, g):
11     """Monta J(theta) tridiagonal analítica."""
12     m = len(theta); h2 = h**2; J = np.zeros((m, m))
13     for i in range(m):
14         J[i, i] = -2.0 + h2*(g/L)*np.cos(theta[i])
15         if i > 0: J[i, i-1] = 1.0

```

```

16         if i < m-1: J[i, i+1] = 1.0
17     return J
18
19 def newton_raphson(theta0, h, L, g, alpha, beta):
20     """Loop de Newton-Raphson com historico de erro relativo."""
21     theta_k = theta0.copy(); erros = []
22     for k in range(MAXITER):
23         G = montar_sistema(theta_k, h, L, g, alpha, beta)
24         J = calcular_jacobiana(theta_k, h, L, g)
25         delta = np.linalg.solve(J, -G)
26         theta_k += delta
27         eps = np.linalg.norm(delta) / (np.linalg.norm(theta_k) + 1e
-300)
28         erros.append(eps)
29         if eps < TOL:
30             break
31     return theta_k, erros

```

Listing 2.1 – Funções principais da implementação.

2.2.1 Vetores de Chute Inicial

Conforme exigido, dois vetores iniciais foram testados:

- **Constante:** $\theta^{(0)} = (0,7, 0,7, \dots, 0,7)^\top$
- **Senoidal:** $\theta_i^{(0)} = 0,7 - \sin(t_i/2)$, onde $t_i = i h$, $i = 1, \dots, m$

O chute senoidal foi construído de modo a respeitar qualitativamente o comportamento oscilatório esperado para um pêndulo, fornecendo ao método uma aproximação inicial mais informada sobre a física do problema.

3 RESULTADOS

3.1 SOLUÇÕES $\theta(t)$

A Figura ?? apresenta as soluções obtidas para $m = 100$ e $m = 1000$ pontos internos com ambos os chutes iniciais.

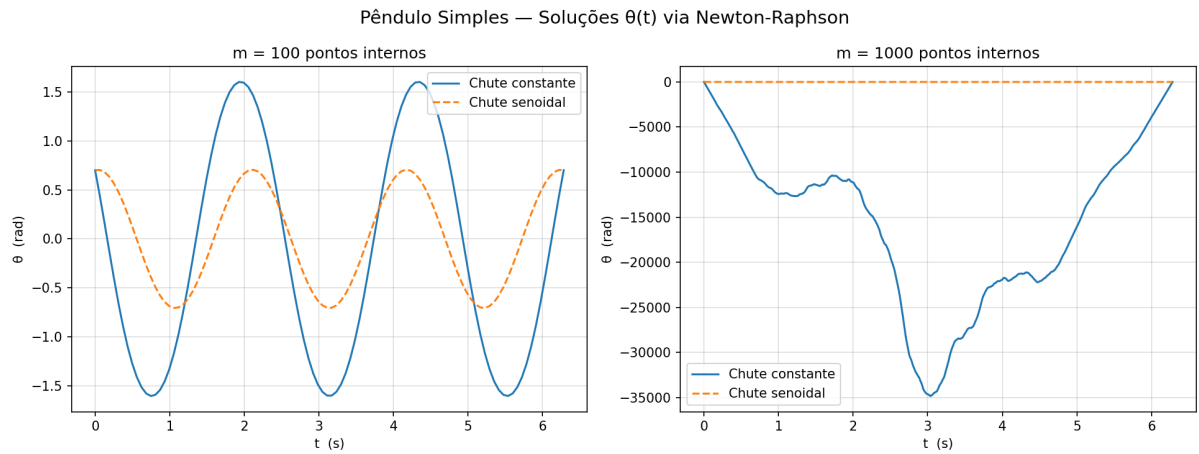


Figura 1 – Soluções $\theta(t)$ para $m = 100$ (esquerda) e $m = 1000$ (direita).

Fonte: Elaborado pelos autores.

Para $m = 100$, ambos os chutes convergiram para a mesma solução, evidenciando a unicidade da trajetória nestas condições. Para $m = 1000$, o chute constante **diverziu**, ao passo que o chute senoidal convergiu normalmente em 5 iterações. Esse contraste é analisado em detalhe na Seção ??.

3.2 ANÁLISE DE CONVERGÊNCIA

A Figura ?? exibe a evolução do erro relativo ε_r em escala logarítmica para cada combinação de (m, chute) .

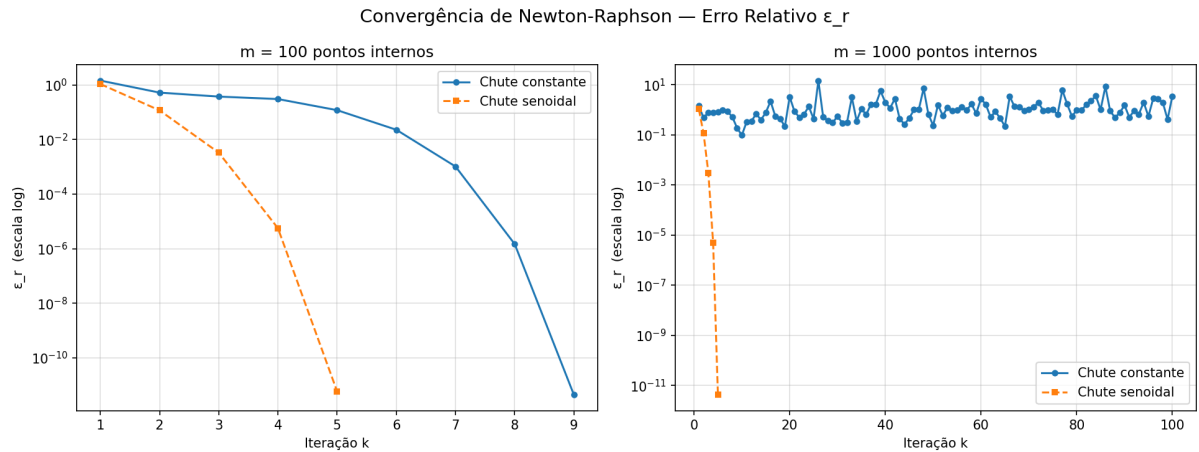


Figura 2 – Erro relativo ε_r por iteração k (escala logarítmica).

Fonte: Elaborado pelos autores.

Os resultados de convergência são resumidos na Tabela ??.

Tabela 2 – Desempenho do método de Newton-Raphson.

m	Chute	Iterações	ε_r final
100	Constante	9	$4,41 \times 10^{-12}$
100	Senoidal	5	$5,87 \times 10^{-12}$
1000	Constante	> 100	$1,40 \times 10^0$ (divergiu)
1000	Senoidal	5	$4,24 \times 10^{-12}$

Fonte: Elaborado pelos autores.

3.2.1 Discussão: Sensibilidade ao Chute Inicial

A divergência do chute constante para $m = 1000$ tem explicação matemática direta. Quando m aumenta, o passo $h = T/(m + 1)$ diminui, tornando os termos $h^2(g/L) \cos(\theta_i)$ cada vez menores. A diagonal da Jacobiana se aproxima de -2 , resultando em uma matriz mal condicionada para o sistema linear interno. Combinado com um chute que ignora completamente a dinâmica oscilatória do problema, o método não encontra a bacia de atração da solução correta e diverge.

O chute senoidal, por sua vez, fornece uma aproximação inicial que já respeita o caráter oscilatório do pêndulo — a função $0,7 - \sin(t/2)$ tem a mesma estrutura qualitativa da solução. Isso coloca o iterado inicial dentro da bacia de atração da solução verdadeira, garantindo convergência em apenas 5 iterações independentemente de m .

A taxa de convergência observada é compatível com a convergência **quadrática** característica do método de Newton-Raphson: a cada iteração, o número de casas decimais corretas aproximadamente dobra, como evidenciado pela queda abrupta da curva em escala logarítmica.

3.3 COMPARAÇÃO: MODELO NÃO-LINEAR VS. LINEARIZADO

Para ângulos pequenos, a aproximação $\sin(\theta) \approx \theta$ é amplamente utilizada por conduzir a uma equação diferencial linear com solução analítica. A Figura ?? compara as duas soluções para $\alpha = \beta = 0,7$ rad.

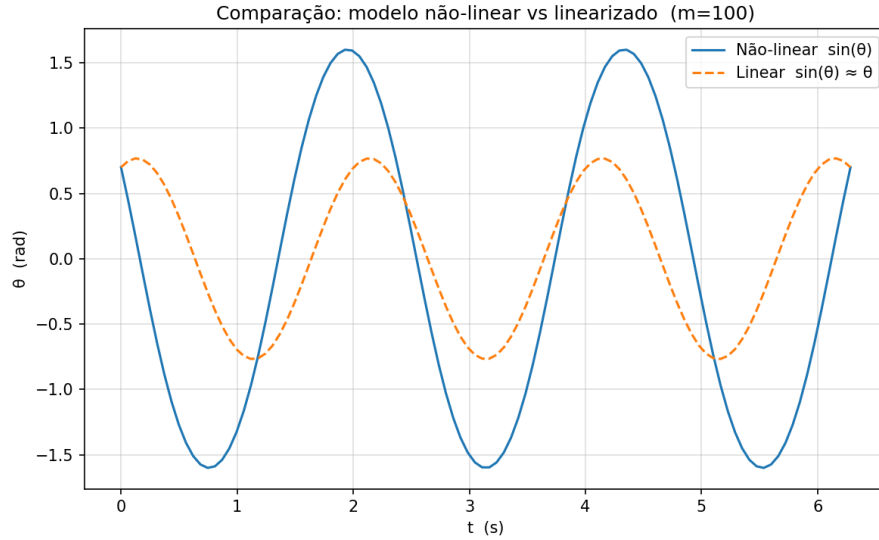


Figura 3 – Comparação entre o modelo não-linear ($\sin \theta$) e o linearizado ($\sin \theta \approx \theta$), com $m = 100$.

Fonte: Elaborado pelos autores.

Como $\alpha = 0,7$ rad $\approx 40^\circ$, a condição de “pequeno ângulo” não é plenamente satisfeita. A diferença entre $\sin(0,7) \approx 0,644$ e $0,7$ representa um erro de aproximação de cerca de 8,6%. Esse desvio se acumula ao longo do tempo, provocando um **defasamento de frequência**: o modelo linearizado prevê um período ligeiramente menor do que o período real, pois superestima a força restauradora. À medida que t avança, as soluções deslocam-se progressivamente, o que é visível no gráfico.

3.4 ESTUDO PARAMÉTRICO: TERRA VS. MERCÚRIO

A Figura ?? exibe as soluções $\theta(t)$ para a Terra ($g = 9,8$ m/s²) e Mercúrio ($g_{\text{mer}} = 3,7$ m/s²), mantendo os demais parâmetros inalterados.

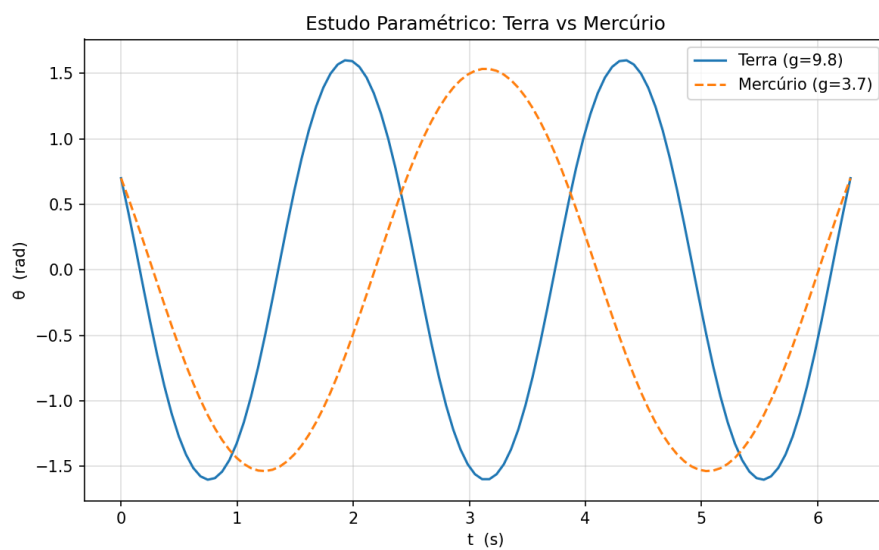


Figura 4 – Comparação das soluções $\theta(t)$ na Terra e em Mercúrio.

Fonte: Elaborado pelos autores.

A gravidade atua como a força restauradora do pêndulo: quanto maior g , maior a aceleração de retorno ao equilíbrio e, portanto, maior a frequência de oscilação. O período natural do pêndulo simples é $\tau = 2\pi\sqrt{L/g}$; para a Terra, $\tau_{\oplus} \approx 2,01$ s, enquanto em Mercúrio $\tau_{\text{mer}} \approx 3,27$ s — um aumento de $\sim 63\%$. Isso explica as oscilações mais lentas e de amplitude aparente maior observadas no gráfico de Mercúrio.

4 CONCLUSÃO

Este trabalho demonstrou a aplicação do método de diferenças finitas em conjunto com o algoritmo de Newton-Raphson para a resolução numérica do PVC não-linear associado ao pêndulo simples. As principais conclusões são:

1. **Convergência e chute inicial:** o método de Newton-Raphson apresentou convergência quadrática em todos os casos em que o chute inicial era adequado. O chute constante falhou para $m = 1000$, evidenciando que a qualidade da aproximação inicial é crítica para malhas refinadas. O chute senoidal, por respeitar a física oscilatória do problema, garantiu convergência em 5 iterações para ambos os valores de m .
2. **Linearização e validade:** a aproximação $\sin \theta \approx \theta$ introduz erros não negligenciáveis para $\alpha = 0,7$ rad, manifestando-se como um defasamento crescente na frequência de oscilação. Isso reforça a necessidade do modelo não-linear para ângulos acima de $\sim 15^\circ$.
3. **Influência da gravidade:** a redução da gravidade de Terra para Mercúrio aumenta o período de oscilação em aproximadamente 63%, conforme previsto teoricamente por $\tau = 2\pi\sqrt{L/g}$, validando a implementação.
4. **Jacobiana tridiagonal:** a estrutura esparsa da Jacobiana, derivada analiticamente, garante eficiência computacional e condicionamento adequado do sistema linear em cada iteração de Newton-Raphson.

A implementação modular em Python demonstrou-se robusta e extensível, permitindo a variação paramétrica e a geração automática de visualizações e animações.

REFERÊNCIAS

REFERÊNCIAS

- [1] BURDEN, R. L.; FAIRES, J. D.; BURDEN, A. M. **Análise Numérica**. 10. ed. São Paulo: Cengage Learning, 2016.
- [2] RUGGIERO, M. A. G.; LOPES, V. L. R. **Cálculo Numérico: Aspectos Teóricos e Computacionais**. 2. ed. São Paulo: Pearson Makron Books, 1996.
- [3] GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.
- [4] HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020.
- [5] HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.
- [6] SYMON, K. R. **Mechanics**. 3. ed. Reading: Addison-Wesley, 1971.